

A Technical Audit of a Wolfram Language Radical-Denesting Program

Algebraic design, branch correctness, search heuristics,
and engineering risks

Source-grounded analysis of the supplied Pasted `text.txt`

5 September 2026

Abstract

The supplied program attempts to simplify nested algebraic radicals by factoring expressions, collecting radical subproblems, and searching for multiplicative changes of variables that make a polynomial greatest common divisor have lower degree. Its central algebraic idea is useful: the minimal polynomial of a suitable multiple of a radical can isolate a simpler polynomial relation over an algebraic coefficient field. This article reconstructs that mechanism, proves the associated candidate-generation and denominator-rationalization identities, and follows the implementation from its public wrappers to its auxiliary tree-processing routines.

The most consequential defect is a loss of the original complex branch: the code replaces a target α by the principal root of α^k when selecting an answer. Explicit examples show a sign reversal for a negative real target, a complex $3/2$ power, and a product of two principal square roots. Separate defects include branch-unsafe exponent manipulation in the custom factorizer, collisions between temporary symbols and user expressions, an incorrect sorting comparator, an unguarded access to an empty candidate list, and a supposed multiplier cap that can allow billions of candidates. Other limitations are heuristic rather than correctness defects: incomplete factorization, degree-based pruning, unproven search coverage, and the absence of a verified denesting objective.

The audit is based on the complete supplied source, official Wolfram documentation, and independently executed symbolic and numerical checks. The uploaded Wolfram program and the proposed Wolfram regression tests were *not* executed in a Wolfram kernel. This distinction is maintained throughout.

Source fingerprint (SHA-256)

693a2825f1e2b8912e55371b90dc8ed115bc
33ee707dd3a4521eb22bcd2ccc93

The archive contains the article, its LaTeX source, the original code,
independent verification scripts and results, and proposed regression tests.

Contents

1	Scope, evidence, and the intended contract	4
1.1	What was actually supplied	4
1.2	What counts as a correct denester	4
1.3	Evidence levels and execution boundary	4
2	Architecture: from an expression to a multiplier search	5
3	The wrapper in detail	6
3.1	Marking and preprocessing radical subexpressions	6
3.2	Combining subproblems	6
3.3	Single-pass versus all-level processing	6
4	The algebra behind the engine	7
4.1	Choice of a root order	7
4.2	A trial multiplier	7
4.3	Why the root-of-unity enumeration is a good idea	8
5	A complete worked example	8
6	Search state, multiplier generation, and stopping	9
6.1	Initial multipliers	9
6.2	History and worklist representation	9
6.3	New multipliers	10
6.4	When the code declares success	10
7	The auxiliary mathematics and data structures	10
7.1	Denominator rationalization: a valid idea	10
7.2	Nesting-height annotations	11
7.3	The custom factorizer	11
8	Critical correctness defect: the original branch is discarded	12
8.1	The faulty equality target	12
8.2	Negative real input: the simplest direct-engine example	12
8.3	A complex rational power admitted by the wrapper	13
8.4	The problem persists even when both outer powers are square roots	13
8.5	Required correction	14
9	Other value-changing defects	14
9.1	Unconditional flattening of powers in <code>fullfactor</code>	14

9.2	Temporary-symbol collisions	15
9.3	Loading the file changes a system symbol	15
10	Defensive-programming and concrete implementation defects	16
10.1	The prime-sorting comparator ignores its second argument	16
10.2	An empty candidate list is accessed unconditionally	16
10.3	The equality test is heuristic, not a certificate	16
10.4	Input and intermediate-result validation is missing	16
10.5	The “terms” extraction is not actually restricted to sums	17
11	Search limitations and resource risks	17
11.1	Equal absolute degrees can hide a useful GCD	17
11.2	Partial factorization is intentionally incomplete	18
11.3	The advertised multiplier cap is not a cap	18
11.4	The subset pass adds another exponential mechanism	18
11.5	Termination and failure status need a real contract	19
11.6	Sorting and simplification have hidden costs	19
12	Consolidated findings and repair priorities	19
13	A safer design that preserves the useful mathematics	20
13.1	Separate proposal generation from proof	20
13.2	Use exact algebraic verification conservatively	21
13.3	Choose a denesting objective explicitly	21
13.4	Make state and resources explicit	21
13.5	A guarded one-multiplier step	22
14	Verification performed and proposed regression testing	22
14.1	Independently executed checks	22
14.2	Wolfram tests supplied but not executed	22
14.3	Tests needed beyond the supplied regression file	23
15	Conclusion	23
	References	23
A	Complete supplied source, with physical line numbers	25
B	Archive contents and reproducibility	34

1 Scope, evidence, and the intended contract

1.1 What was actually supplied

The attachment is a Wolfram Language program, not a prose specification. It defines `Strad`, `DenestRadicals3`, `denest11`, several mathematical helpers, and a custom factorization subsystem. The original byte stream is preserved in `original.wl`; its 585 physical lines are reproduced in Appendix A. All source-line references in this article refer to that file. The attachment provides no author attribution, version requirement, test suite, benchmark corpus, formal input contract, or completeness theorem. None is inferred here.

The public-looking entry points are

```
Strad[expr, alllevels, func]
DenestRadicals3[expr, alllevels : False, func : denest11]
denest11[problem, forcedmultipliers : 0]
```

The first signature above describes forwarding, not a separately declared three-argument definition: the actual `Strad[eaf__]` accepts a nonempty sequence and passes it onward (source lines 1–5). The `forcedmultipliers` argument belongs to `denest11`; it is not the second argument of the wrapper.

The wrapper recognizes powers whose exponents are exact `Rational` objects and whose bases satisfy `AlgebraicQ`. The implementation of that predicate is `Element[num, Algebraics]` (source lines 453–454). The code therefore appears intended primarily for explicit, exact algebraic numbers written using radicals. However, its signatures do not enforce that domain, and `Factorc[expr]` is applied to the entire input *before* any algebraicity test. This matters for the symbolic counterexamples below.

1.2 What counts as a correct denester

For an exact scalar algebraic input α , a successful simplifier should at least return an expression β satisfying

$$\beta = \alpha \tag{1}$$

under the language’s branch conventions. A *denesting* success should additionally improve an explicitly chosen representation criterion. Equality alone is not a proof of denesting; a lower minimal-polynomial degree for a transformed auxiliary number is not a proof of denesting either.

For example, the same algebraic number can be written as a nested radical, a sum of unnested radicals, or a `Root` object. A conversion to a `Root` object can eliminate all visible `Power` nodes without providing an unnested radical formula. Any practical contract must state which of these output forms are allowed.

1.3 Evidence levels and execution boundary

Evidence	Meaning in this article
Source fact	A statement directly established by the supplied definitions, indices, conditions, or assignments.
Algebraic proof	A mathematical consequence proved here independently of a particular Wolfram kernel’s evaluation choices.
Executed check	A computation actually run with SymPy 1.14.0 or mpmath 1.3.0. These checks do not execute or emulate the uploaded program.

Evidence	Meaning in this article
Predicted trace	A Wolfram-level behavior deduced from the source and documented semantics; the corresponding regression test is supplied but was not run here.
Conditional risk	A failure mode whose occurrence depends on an input, a callback, resource exhaustion, or solver behavior not established by a complete runtime trace.

No local Wolfram executable was available, and no connected Wolfram computation service was used. Consequently, this report does not claim a Mathematica version, end-to-end timing, an observed Wolfram message transcript, or a passing Wolfram test count. Some concrete examples can be simplified automatically before reaching a faulty path in particular releases. Their mathematical role is to expose the invalid transformation; the regression suite determines runtime manifestation.

Official documentation is used only to establish language semantics and built-in function behavior. The algorithmic interpretation, counterexamples, proofs, and recommendations are the analysis developed from the supplied source.

2 Architecture: from an expression to a multiplier search

Routine	Lines	Responsibility
Strad	1–3	Forward arguments while suppressing calls to <code>Print</code> .
DenestRadicals3	4–76	Preprocess the expression, mark radicals, combine subproblems, dispatch to a denester, and substitute results.
denest11	78–441	Choose a root order, generate multipliers, manage search history and a mutable worklist, and retain a proposed solution.
trymultiplier	107–155	Compute a minimal polynomial and possibly a lower-degree GCD; solve and select a candidate. Local to <code>denest11</code> .
newmultipliers	159–211	Generate additional multipliers using discriminant factors or a product of previously useful multipliers. Also local.
RationalizeDenominator	443–451	Replace the reciprocal of an algebraic denominator by a polynomial in that denominator.
AlgebraicQ	453–454	Ask whether an expression belongs to the algebraic numbers.
nestdepths	456–466	Recursively annotate an expression tree with a custom nesting height.
nestdepthsort	468–480	Flatten and group those annotations by height.
maxnestdepth	482–498	Extract the relevant maximum-height frontier.
ComplexitySort	500–504	Sort first by <code>LeafCount</code> , then by magnitude.
Factorrc, fullfactor, combine	506–584	Encode products and sums as nested lists, extract common factors, and reconstruct an expression.

The principal flow is

Strad $\longrightarrow$ DenestRadicals3 $\longrightarrow$ denest11
 $\searrow$ factorization and tree helpers $\searrow$ multiplier trials and new search frames.

The auxiliary factorizer is not a harmless formatting pass. It changes the actual expression that every later stage sees. A correctness check performed only inside `denest11` cannot repair a value already changed by this preprocessing.

3 The wrapper in detail

3.1 Marking and preprocessing radical subexpressions

The value of `alllevels` chooses `ReplaceAll` unless it is literally `True`; only that exact value chooses `ReplaceRepeated` (source lines 7–10). Thus a typo or non-Boolean value silently selects the default mode rather than producing an option error.

For each matching rational power, the wrapper splits its base into a numerical content and a remaining factor using `FactorTermsList`. It further processes the remainder through `Factorrc`. The numerical content is decomposed using the GCD of its real and imaginary parts and then `FactorInteger` (source lines 16–25). The local predicate `SumQ` is not merely a test of whether the outer head is `Plus`: it searches for a sum at any level and also recognizes an explicit complex number with nonzero real part (source lines 13–15).

Factors identified by this predicate are multiplied together and passed through `ComplexExpand`. Factors with numerically negative real part are negated, while a separate sign accumulator records the compensating sign. Finally, the factors are rationalized individually, multiplied back together, and simplified (source lines 26–40).

There is an important distinction here. The sign accumulator preserves the product algebraically: replacing a factor a by $-a$ while multiplying the accumulated sign by -1 does not change the product. An inaccurate sign classification can make the heuristic less useful, but it is not by itself the branch-loss defect proved later. Similarly, `ComplexExpand` assumes unspecified variables are real, but on the intended explicit algebraic-number domain there should be no such variables to assume real [15]. It is therefore inappropriate to attribute every complex-branch problem to that built-in.

The processed radical becomes an internal object `f[base, exponent]`. The symbol `f` is localized by `Block`, not made fresh by `Module`; Section 9.2 explains why this distinction matters.

3.2 Combining subproblems

Two marked factors are combined into one marked *product of their powers* when both bases contain a `Power` node (source lines 43–47). This is not the naive identity $(ab)^r = a^r b^r$: the code actually retains the product $(\text{rad1}^{\text{pow1}}) * (\text{rad2}^{\text{pow2}})$ inside the marker. The grouping itself therefore preserves the value.

Next, two-argument markers are changed into one-argument markers containing the powered expression (source lines 48). Another rule combines marked complex values when both real and imaginary parts are numerically nonzero at precision 10 (source lines 49–54). Again, the product is retained. The danger is downstream: the combined product need not be a principal k th root, while the engine later treats it as one.

3.3 Single-pass versus all-level processing

The wrapper collects distinct occurrences of `f[_]` throughout the expression and logs them (source lines 55–57). In default mode, it constructs a substitution rule for each collected problem using the selected callback `func`, then simplifies the result (source lines 58–61).

In all-level mode, inner markers may occur inside outer markers. The code cycles through the problem list until it finds a marker with no unresolved marker inside it. It replaces that entry by a rule, propagates the rule through

the remaining problems, and eventually applies all rules to the main expression (source lines 62–73). This is a manually implemented bottom-up dependency resolution.

For a finite, well-formed marker tree and a callback that returns marker-free expressions, the intended process can make progress by resolving an innermost marker. It is not accurate to call this loop intrinsically nonterminating. Nevertheless, it has no explicit check that a full cycle resolved something, no callback-result contract, and no protection against a user expression that already uses the same marker symbol.

The meaning of `alllevels -> True` is thus structural: process the marked levels of the supplied expression. It does not establish that all mathematically possible denestings are found, nor does it prove that every newly introduced radical will itself be denested to a fixed point.

4 The algebra behind the engine

4.1 Choice of a root order

After a syntactic nested-power test, `denest11` asks `maxnestdepth` for rational powers at the maximum relevant nesting height. It sets

$$k = \text{lcm}\{\text{denominators of those exponents}\}, \quad \text{outerpower} = \frac{1}{k}, \quad A = \alpha^k, \quad (2)$$

where α is the original problem (source lines 84–94). The intended effect is to remove the outer radical layer. This k is selected from a syntactic frontier; it is not a proven minimal field-theoretic root order, nor is it necessarily the LCM of every rational exponent anywhere in the expression.

A second quantity, `extrapower`, is calculated from deeper levels and printed (source lines 95–103). It is never subsequently used to change the target, the multiplier search, or candidate validation. The program therefore does not implement an additional lifting step merely because it prints an “extra root.”

4.2 A trial multiplier

For a nonzero algebraic multiplier m , a trial constructs

$$\beta = (mA)^{1/k}, \quad P_m(X) = \text{minpoly}_{\mathbb{Q}}(\beta). \quad (3)$$

Here and below rational powers have their principal complex meaning. `MinimalPolynomial[s]` legitimately returns a pure-function representation; calling that result on `arg1` is supported syntax, not a missing polynomial variable bug [2]. The code records $d_m = \deg P_m$ (source lines 111–114).

For selected trials, it computes

$$G_m(X) = \gcd_{K[X]}(P_m(X), X^k - mA), \quad (4)$$

using `Extension -> Automatic` (source lines 115–120). The field K contains the algebraic numbers present in the polynomial coefficients. The representation of those coefficients can expose a larger field than the smallest field generated by their evaluated values; it is safer to describe K this way than to assert that it is always exactly $\mathbb{Q}(mA)$. The documented purpose of the option is to include algebraic coefficients in the coefficient field [3].

Proposition 4.1 (Why the GCD is meaningful). *Assume $A, m \neq 0$ are algebraic, $k \geq 2$, and the operations are exact. Then $1 \leq \deg G_m \leq k$. Every root t of G_m satisfies $t^k = mA$.*

Proof. The chosen value β is a common root of the two polynomials in (4). Its minimal polynomial over K divides both, so their GCD is nonconstant. The GCD divides $X^k - mA$, giving the degree bound and the asserted root equation. $\square$

When $\deg G_m < k$, the code regards the trial as having found a denesting. A linear GCD is especially useful: it directly expresses one root in the coefficient field. A quadratic or cubic GCD may provide a smaller equation, but its solutions still need to be inspected to determine whether they improve the requested radical representation.

4.3 Why the root-of-unity enumeration is a good idea

The code solves $G_m(t) = 0$, divides the resulting roots by the principal $m^{1/k}$, and multiplies by every k th root of unity (source lines 121–124).

Proposition 4.2 (Candidate coverage for a fixed successful GCD). *Let t be any root of G_m and $m \neq 0$. Then the set*

$$\left\{ \frac{t}{m^{1/k}} \exp\left(\frac{2\pi i j}{k}\right) : j = 0, 1, \dots, k-1 \right\} \quad (5)$$

is exactly the set of k th roots of A . In particular, it contains the original α whenever $A = \alpha^k$.

Proof. Since $(m^{1/k})^k = m$, the k th power of $t/m^{1/k}$ is A . Multiplication by the k roots of unity gives all roots of $X^k - A$, which are distinct because $A \neq 0$ and the characteristic is zero. $\square$

This argument does *not* require the generally false identity $(mA)^{1/k} = m^{1/k}A^{1/k}$. Enumerating the root-of-unity orbit is precisely what can compensate for the choice of principal roots. The implementation already creates the right type of candidate set. Its major mistake is selecting from that set against $A^{1/k}$ instead of against the original α .

5 A complete worked example

Consider

$$\alpha = \sqrt{5 + 2\sqrt{6}} = \sqrt{2} + \sqrt{3}. \quad (6)$$

Here $k = 2$ and $A = 5 + 2\sqrt{6}$. At $m = 1$,

$$P_1(X) = X^4 - 10X^2 + 1. \quad (7)$$

Over $K = \mathbb{Q}(\sqrt{6})$,

$$G_1(X) = X^2 - 5 - 2\sqrt{6}. \quad (8)$$

The GCD still has degree $k = 2$, so this trial does not yield a smaller equation. It does provide the first finite degree record for the search.

At $m = 2$, however,

$$\beta = \sqrt{10 + 4\sqrt{6}} = 2 + \sqrt{6}, \quad (9)$$

and therefore

$$P_2(X) = X^2 - 4X - 2, \quad (10)$$

$$G_2(X) = X - 2 - \sqrt{6}. \quad (11)$$

The selected root gives

$$\frac{2 + \sqrt{6}}{\sqrt{2}} = \sqrt{2} + \sqrt{3}. \quad (12)$$

All roots and signs agree with the original positive target in this example.

This illustrates why a multiplier helps: it moves the transformed root into a smaller absolute-degree situation and, more importantly, makes the GCD linear over the available coefficient field. The degree drop is a useful clue here, but the linear GCD is the actual algebraic information used to construct the expression.

The discriminant of P_1 is

$$\text{disc}(P_1) = 147456 = 2^{14}3^2. \quad (13)$$

Thus the discriminant-based generator can discover prime multipliers 2 and 3 in this example. This successful illustration is not a theorem that discriminant primes suffice for arbitrary inputs.

The displayed minimal polynomials, GCDs, discriminant, and identity were checked independently with SymPy. The order in which the uploaded program reaches $m = 2$ was not measured in a Wolfram execution.

6 Search state, multiplier generation, and stopping

6.1 Initial multipliers

When `forcedmultipliers` is a list, the code uses that list to seed the search. Otherwise it expands a rationalized version of A , removes selected rational coefficients from the putative terms, and forms complementary powers of radicals (source lines 214–249). For a term $b^{u/v}$ with $u \neq 0$, the intended local calculation is

$$\left(b^{u/v}\right)^{(v-u)/u} \approx b^{(v-u)/v}, \quad (14)$$

so that multiplication by the original term would remove its fractional exponent at a formal level. Complex branch laws mean this mnemonic is not an unrestricted identity. Because these expressions are merely trial multipliers, their equality to some “ideal” multiplier is not required for soundness, provided the multiplier is nonzero and the eventual result is certified against the original input.

The list starts with multiplier 1 in automatic mode. A list passed as `forcedmultipliers` does not strictly prohibit later generated multipliers: progress-triggered insertion is still present in the main loop. Its practical meaning is closer to “initial multipliers” than to “the only multipliers that may be tried.”

6.2 History and worklist representation

The history starts with a sentinel

```
{0, Infinity, Infinity}
```

and records triples `{multiplier, minpoly, degree}`. The history insertion logic orders records by decreasing degree, placing the smallest degree at the end (source lines 380–388). The sentinel makes the first finite-degree trial count as progress without a separate initialization branch.

The so-called `multiplierstack` is an ordered, mutable worklist rather than an ordinary last-in-first-out stack. A realized frame has the shape

```
{{doFullSet, generatingAttempt}, multiplierList}
```

and a deferred frame stores a function and its arguments until it is reached (source lines 253–267, 346–348). During processing, entries in a multiplier list change from scalar multipliers into pairs `{multiplier, degree}` (source lines 292).

If the current frame is not a full-set frame, a newly generated frame is inserted immediately after the completed prefix and before any remaining suffix. In a full-set frame, additional work is scheduled later, using degree and expression size as ordering clues (source lines 350–375). This is a reasonable search-management idea, but the same nested-list structure carries several different schemas. Its invariants are implicit, and no shape validation protects the many positional accesses.

6.3 New multipliers

In the usual branch, the generator takes a partial factorization of the discriminant of a recorded minimal polynomial, discards factors that do not pass `PrimeQ`, and uses selected primes to form divisors of

$$\left(\prod_{j=1}^r p_j \right)^{k-1}. \quad (15)$$

These divisors, together with additional discovered primes, are multiplied by the previous multiplier (source lines 169–199).

In the alternative branch, where observed degrees have a GCD smaller than either one, the generator tries the product of the two recorded multipliers (source lines 201–207). The code supplies no field-theoretic proof that this product will realize the numerical GCD of the degrees. It should be read as a search heuristic.

After the first automatic pass, the program also builds products of subsets of some promising initial multipliers, then reduces certain exponent patterns (source lines 396–426). Several transformations in this block are not general complex identities. Again, that is not automatically a returned-value bug: arbitrary nonzero algebraic multipliers are legitimate search proposals. It does limit the meaning of any claim that the generated list represents all equivalent cases.

6.4 When the code declares success

An attempt with a non-list last component is interpreted as carrying a solution. The solution is simplified and sometimes returned immediately, depending on its absolute degree relative to an earlier record (source lines 309–324). This result protocol uses the shape of the last element as a type tag. It assumes that a valid solution is scalar and that no malformed expression has accidentally occupied that slot.

The program does not compare the new expression’s radical nesting height with that of the input before declaring success. Later “additional denesting” attempts can replace an earlier solution without an explicit objective proving that the replacement is a better radical expression. `Simplify` is used, but no custom denesting complexity function is supplied.

7 The auxiliary mathematics and data structures

7.1 Denominator rationalization: a valid idea

Write an input as N/d with nonzero algebraic denominator d , and let P be its minimal polynomial. Polynomial division over a field containing d gives

$$P(X) = (X - d)Q(X), \quad (16)$$

since $P(d) = 0$. Evaluation at zero yields

$$P(0) = -dQ(0), \quad \frac{1}{d} = -\frac{Q(0)}{P(0)}. \quad (17)$$

Because $d \neq 0$, its irreducible minimal polynomial cannot have zero constant term. The code implements exactly this construction (source lines 443–451).

The argument does not require a monic polynomial. This is relevant because `MinimalPolynomial` uses a primitive integer polynomial with positive leading coefficient, which may be nonmonic [2]. For example,

$$\frac{1}{1 + \sqrt{2}} = \sqrt{2} - 1, \quad (18)$$

$$\frac{1}{\sqrt{2}/3} = \frac{3\sqrt{2}}{2}, \quad (19)$$

$$\frac{1}{1 + i\sqrt{2}} = \frac{1 - i\sqrt{2}}{3}. \quad (20)$$

All three identities, including the nonmonic case, were independently checked. The variable called `conjugate` actually holds a minimal-polynomial function, not a single algebraic conjugate.

The mathematical formula is sound on its stated domain. The implementation still has a temporary-symbol collision discussed below and lacks explicit handling of zero, symbolic or nonalgebraic denominators, failed polynomial operations, and resource limits.

7.2 Nesting-height annotations

For a predicate p on expression nodes, the intended height is

$$h_p(e) = \mathbf{1}_{p(e)} + \max(0, \{h_p(c) : c \text{ is a child of } e\}). \quad (21)$$

The code’s common predicate is recognition of `Power[_ , _Rational]`. `nestdepths` stores the original expression, all child annotations, and its computed height in a recursively nested list (source lines 457–466). `nestdepthsort` rewrites that tree into groups of equal height, while `maxnestdepth` prunes below the maximum and extracts a frontier (source lines 469–498).

This is a representation measure, not an invariant of the algebraic number. Algebraically equal expressions can have different heights. The annotation format also embeds original expression trees at many nodes, which can cause substantial memory and rewriting overhead on deeply nested inputs. Physical sharing may reduce actual memory in a particular kernel, so an unconditional measured quadratic-memory claim would be too strong; the structural duplication risk is nonetheless clear.

The base test uses `Depth[expr] === 1`, not an explicit empty-child policy. A non-atomic expression with no arguments can make the child maximum an empty maximum, rather than the intended zero in (21). This is an edge-domain risk, not a demonstrated failure on ordinary scalar radical expressions.

The entry test in `denest11` looks for arbitrary nested `Power` nodes, whereas the later height computation counts only rational exponents. Moreover, `MemberQ[... , Infinity]` searches levels 1 and deeper, not level 0 [14]. A root-level power used as the immediate base of another power can therefore be missed by that inner test when it has no further power inside. The gate and the subsequent arithmetic should be brought under one explicit radical grammar rather than treated as interchangeable tests.

7.3 The custom factorizer

`fullfactor` begins with `FactorList`. It then repeatedly processes factor/exponent pairs. Numerical rational factors are prime-factorized; sums are recursively decomposed; a factor that is itself a power has its exponent multiplied by the outer stored exponent (source lines 519–548). `combine` merges repeated bases inside each

summand, finds bases common to every summand, subtracts their minimum exponents, and stores the extracted factors separately (source lines 554–584).

`Factorrc` reconstructs powers, products, and sums using rewrite rules that recognize the depth and shape of nested lists (source lines 507–516). Tagged structures such as `ProductNode`, `SumNode`, and `PowerNode` would make this representation considerably easier to validate. As written, ordinary list shape carries semantic meaning, and the final `Flatten[...][[1]]` idiom assumes the reconstruction has produced exactly one meaningful expression.

The local `j < 5` limit in `fullfactor` is an unexplained pass bound. It may leave some factoring opportunities unexplored. The presence of a remaining factor does not by itself prove a wrong numerical answer; incomplete factoring must be distinguished from the genuinely invalid exponent laws below. Also, the five-pass bound does not bound recursive calls on sums or the larger denesting search.

8 Critical correctness defect: the original branch is discarded

8.1 The faulty equality target

The code sets $A = \alpha^k$, but its candidate filter is

```
Select[possibilities,
PossibleZeroQ[# - radicand^outerpower] &]
```

(source lines 125–127). In mathematical notation it tests

$$\gamma = A^{1/k} = (\alpha^k)^{1/k}, \quad (22)$$

not $\gamma = \alpha$.

Proposition 8.1 (Exact description of the phase loss). *Let $\alpha \neq 0$, write $\alpha = \rho e^{i\theta}$ with $\theta = \text{Arg } \alpha \in (-\pi, \pi]$, and choose $n \in \mathbb{Z}$ so that $k\theta - 2\pi n \in (-\pi, \pi]$. Then*

$$(\alpha^k)^{1/k} = \alpha e^{-2\pi i n/k}. \quad (23)$$

In particular, the reset preserves the target only when α is in the principal k th-root sector $-\pi/k < \text{Arg } \alpha \leq \pi/k$.

Proof. The principal logarithm of α^k is $k \log \rho + i(k\theta - 2\pi n)$. Divide it by k and exponentiate. $\square$

Wolfram rational powers use the principal complex root, so this is the relevant branch convention [5, 6]. For $k = 2$, the reset can change the sign. For higher k , it can multiply the target by a nontrivial root of unity.

8.2 Negative real input: the simplest direct-engine example

Take

$$\alpha = -\sqrt{3 + 2\sqrt{2}} = -(1 + \sqrt{2}). \quad (24)$$

The engine chooses $k = 2$ and $A = 3 + 2\sqrt{2}$. With $m = 1$, the principal root of A is $1 + \sqrt{2}$, whose minimal polynomial is

$$P(X) = X^2 - 2X - 1. \quad (25)$$

The GCD over $\mathbb{Q}(\sqrt{2})$ is $X - 1 - \sqrt{2}$. The enumerated candidate set contains both signs, but the filter selects $1 + \sqrt{2}$, not the negative original input.

A direct reproducer is

```
denest11[-Sqrt[3 + 2 Sqrt[2]], {1}]
```

The predicted incorrect result is $1 + \sqrt{2}$. A lone negative sign outside a radical can remain outside the marker in the public wrapper, so this direct-engine example alone would not prove that the wrapper always mishandles negatives. The next two examples address genuinely complex wrapper subproblems.

8.3 A complex rational power admitted by the wrapper

Let

$$z = -1 + 2i\sqrt{2} = (1 + i\sqrt{2})^2. \quad (26)$$

The principal square root of z is $1 + i\sqrt{2}$, and

$$\alpha = z^{3/2} = (1 + i\sqrt{2})^3 = -5 + i\sqrt{2}. \quad (27)$$

Its real part is negative, so it is not in the principal square-root sector. The code's $k = 2$ produces

$$A = \alpha^2 = 23 - 10i\sqrt{2}, \quad A^{1/2} = 5 - i\sqrt{2} = -\alpha. \quad (28)$$

For $m = 1$, the wrong target has minimal polynomial and GCD

$$P(X) = X^2 - 10X + 27, \quad G(X) = X - 5 + i\sqrt{2}. \quad (29)$$

Thus a successful run through the stated trial and filter selects the opposite value. The relevant public call is

```
DenestRadicals3[(-1 + 2 I Sqrt[2])^(3/2)]
```

The mathematics of (27), the square, the polynomial, and the linear GCD was independently verified. At 30 displayed digits the two values are

$$\begin{aligned} \alpha &= -5 + 1.41421356237309504880168872421 i, \\ A^{1/2} &= 5 - 1.41421356237309504880168872421 i. \end{aligned}$$

These are not rounding variants; they are exactly negatives of each other.

8.4 The problem persists even when both outer powers are square roots

Define

$$u = \sqrt{-1 + 2i\sqrt{2}} = 1 + i\sqrt{2}, \quad (30)$$

$$v = \sqrt{-2 + 2i\sqrt{3}} = 1 + i\sqrt{3}. \quad (31)$$

Both are principal square roots, yet their product is

$$\alpha = uv = 1 - \sqrt{6} + i(\sqrt{2} + \sqrt{3}), \quad (32)$$

which has negative real part and positive imaginary part. The wrapper's product-combination rule can send exactly this product to the engine. Squaring gives

$$A = 2 - 4\sqrt{6} - i(4\sqrt{2} + 2\sqrt{3}). \quad (33)$$

Its principal square root is $-\alpha$. Independently computed values for the wrong target are

$$P(X) = X^4 + 4X^3 + 4X^2 + 48X + 144, \quad (34)$$

$$G(X) = X + 1 - \sqrt{6} + i(\sqrt{2} + \sqrt{3}). \quad (35)$$

The GCD is linear over a field containing $i\sqrt{2}$ and $i\sqrt{3}$. The corresponding public reproducer is

```
DenestRadicals3[
  Sqrt[-1 + 2 I Sqrt[2]] Sqrt[-2 + 2 I Sqrt[3]]]
```

Its intended value is approximately $-1.4494897427831780982 + 3.1462643699419723423 i$; the reset target is its negative. This exposes the interaction between otherwise value-preserving subproblem grouping and the engine's invalid assumption about the grouped target.

8.5 Required correction

The original target must be passed to `trymultiplier` and used in the final filter. A suitable exact algebraic acceptance test is

```
TrueQ[Quiet[Check[RootReduce[candidate - original] == 0, False]]]
```

subject to explicit exact-algebraic input validation and a resource budget. `RootReduce` provides canonical algebraic reduction and can establish exact equality of algebraic numbers [4]. Returning the original expression or an explicit failure when certification is unavailable is safer than accepting a merely plausible match.

The root-of-unity enumeration should be retained. Replacing the wrong filter by a correct one does not require guessing the phase numerically: the original target is already in the candidate orbit under the assumptions proved in Section 4.

9 Other value-changing defects

9.1 Unconditional flattening of powers in `fullfactor`

The branch at source lines 527–531 replaces a factor/exponent pair representing $(b^r)^s$ by the pair representing b^{rs} . The stored outer exponent s can be rational, because earlier processing has already extracted exponents from radical nodes. Hence this is not limited to the safe case of raising a power to an integer.

For an unrestricted symbol z ,

$$\sqrt{z^2} \neq z \quad (36)$$

in general. Setting $z = -1$ gives $1 \neq -1$. A symbolic source trace for `Factorc[Sqrt[z^2]]` reaches the exponent multiplication $2 \cdot \frac{1}{2} = 1$ and can return z . The proposed regression test evaluates the transformed expression at $z = -1$, so the erroneous transformation cannot be hidden by comparing two expressions only at a positive sample point.

There is a second branch-law obligation in `combine`: extracting a common factor from a sum and multiplying its exponent by `pow` reconstructs a form like

$$(cS)^s \mapsto c^s S^s. \quad (37)$$

For $c = S = -1$ and $s = 1/2$, the left side is 1 and the right side is -1 . Appropriate positivity or argument conditions are needed; the data structure does not record or check them.

The wrapper's initial `Factorc[expr]` means an algebraicity guard on later radicals does not protect arbitrary symbolic input from these transformations. Even on a restricted numeric domain, every factoring rewrite used as an equality must be proved branch-safe or independently checked. These are distinct defects from the principal-root reset in the engine.

9.2 Temporary-symbol collisions

`Block` localizes values while retaining symbol identities; `Module` creates fresh local symbols [7, 8]. That difference is material in a program that constructs and rewrites symbolic expressions.

The marker `f` can consume a user function. Suppose Global '`f`' is unassigned and consider

```
DenestRadicals3[f[1], False, Identity]
```

The wrapper's `Block[{f, ...}]` does not give the marker a new name. Its `Cases[mod, f[_], {0, Infinity}]` therefore recognizes the user's `f[1]` as an internal problem. The callback is applied to 1 and the marker is replaced by 1. This predicted trace changes a user expression that contains no radical at all. The same identity collision can interfere with dependency resolution in all-level mode.

The rationalizer's `x` can erase a numerator. For an unassigned Global '`x`', examine

```
RationalizeDenominator[x/(1 + Sqrt[2])]
```

The local polynomial indeterminate is also named `x`. Source lines 448–450 apply `/.` `x -> 0` to the *entire product* of the numerator and the polynomial quotient. The user's numerator is therefore replaced by zero as well. The predicted result is 0 instead of $x(\sqrt{2} - 1)$.

This second example does not require questioning the rationalization identity: the denominator is a perfectly valid nonzero algebraic number. The error lies in using the same symbol for user data and a temporary polynomial variable, and in the scope of the substitution.

Use a private package context and `Module` or a fresh `Unique` symbol for markers and polynomial variables. Merely renaming `f` to a longer global name reduces accidental collisions but does not provide lexical hygiene.

9.3 Loading the file changes a system symbol

The second source line executes `Unprotect[Print]`. No matching restoration of its previous protection state occurs anywhere in the file. This is a load-time global side effect, not just a temporary consequence of calling `Strad`. `Unprotect` removes the `Protected` attribute; ordinary package loading should not leave unrelated system symbols in a different state [16].

A better design is a private logging function or an explicit verbosity option. Expensive debug computations should be placed inside the verbosity branch. Merely replacing `Print` by `Null` & suppresses output, but ordinary function argument evaluation can still compute expressions supplied as log arguments, including the debug `PolynomialGCD` at source line 207 [12].

10 Defensive-programming and concrete implementation defects

10.1 The prime-sorting comparator ignores its second argument

The comparator at source line 181 is

```
#1[[1]] <= #[[2]] &
```

The shorthand `##` means `##1`, not `##2` [12]. On pairs `{prime, exponent}`, this compares the first pair's prime with its own exponent. It never inspects the second pair.

For example, both comparisons of `{2, 1}` and `{3, 1}` return `False`, in either direction. The ordering is not a valid numerical ordering of the prime factors. Because the next stage assumes an ordered sequence when it tests ratios of adjacent primes, the typo can alter which multipliers are retained.

The likely intended repair is

```
SortBy[primes, First]
```

or `##1[[1]] <= ##2[[1]] &`. The objection is the wrong slot, not the use of a non-strict comparison: Wolfram ordering functions can legitimately use such comparisons [11].

10.2 An empty candidate list is accessed unconditionally

After filtering candidates, the code handles the case `Length[result] > 1`, but not `Length[result] == 0`. It then reads `result[[1]]` (source lines 125–134). An empty result therefore creates a `Part` failure rather than a controlled unsuccessful trial. The malformed last component may then be mistaken for a scalar solution by the shape-based success test.

Under ideal exact operations, complete solving, a valid nonzero multiplier, and a sound equality test, the root-of-unity argument ensures a matching root exists. That theorem does not excuse the missing guard: invalid inputs, unevaluated solver results, and heuristic equality decisions can violate those operational assumptions. A zero forced multiplier is a direct stress case, because it creates division by $0^{1/k}$ in the candidate construction.

At minimum, require a nonempty validated candidate list before selecting an entry; otherwise return a typed unsuccessful attempt or `Failure`.

10.3 The equality test is heuristic, not a certificate

`PossibleZeroQ` is documented as using basic symbolic and numerical methods to suggest whether an expression is zero [9]. It is appropriate as an inexpensive screening test, but not as the sole proof of exact branch identity. In this code it also tests against the wrong target. These are separate problems: substituting `RootReduce` without changing the target would certify the wrong branch more reliably, not fix the algorithm.

10.4 Input and intermediate-result validation is missing

The main functions have broad patterns, but their internals assume explicit exact algebraic scalars, nonzero multipliers, positive integer root orders at least 2, well-formed polynomial results, and lists of explicit solutions. These assumptions are not enforced.

The local `trymultiplier` definition only matches `Rational[1, _Integer]` for `outerpower` (source lines 107–108). If root-order extraction produces 1, a noninteger, or an unevaluated object, the call can remain

unevaluated while later code still performs positional accesses on `attempt`. The source does not establish that every input passing its initial, broader nested-power test will produce a valid order.

Lists supplied as forced multipliers are not checked for zero, approximate values, transcendentals, symbols, or duplicates. Built-ins such as `MinimalPolynomial`, `FactorInteger`, `PolynomialGCD`, and `Solve` are followed by indexing and arithmetic without a consistent `Check`, validation, or failure protocol. An empty forced list itself is not necessarily an error: it can simply yield no trial and leave the input unchanged. The problem is that these cases have no explicitly documented status distinction.

10.5 The “terms” extraction is not actually restricted to sums

The initial-multiplier code uses

```
List @@ Expand[RationalizeDenominator[radicand]]
```

(source lines 224–229). Replacing a head by `List` extracts summands only when the expanded expression has head `Plus`. A `Times` expression yields its factors; a `Power` expression yields its base and exponent. An atom is not converted into the desired singleton list. This makes the heuristic dependent on an accidental outer head and can propagate unexpected shapes.

A precise decomposition is

```
summands[e_] := If[Head[e] === Plus, List @@ e, {e}]
```

with a fresh private definition and separate handling of any desired factor analysis. The current error mainly affects search quality and robustness; it does not by itself establish a false equality in the returned expression.

11 Search limitations and resource risks

11.1 Equal absolute degrees can hide a useful GCD

The expensive GCD is attempted only when

```
degree < best || GCD[degree, best] < best
```

(source lines 115). If a new trial has the same absolute degree as the best known trial, this test fails. Yet the degree of the GCD over the coefficient field is not determined by that absolute degree alone.

An explicit construction uses $A = 5 + 2\sqrt{6}$ from Section 5. The $m = 1$ trial has absolute degree 4 and a degree-2 GCD. Now set

$$\beta = 1 + \sqrt{2} + \sqrt{3}, \quad (38)$$

$$m = \frac{\beta^2}{A} = 6 - 2\sqrt{6} - 2\sqrt{2} + 2\sqrt{3} > 0. \quad (39)$$

The transformed principal root is $\sqrt{mA} = \beta$. Its minimal polynomial is

$$P_m(X) = X^4 - 4X^3 - 4X^2 + 16X - 8, \quad (40)$$

still of absolute degree 4. Over $\mathbb{Q}(\sqrt{2}, \sqrt{3})$, however,

$$G_m(X) = X - 1 - \sqrt{2} - \sqrt{3} \quad (41)$$

is linear. The gate rejects this trial whenever `best == 4`, before looking for the useful GCD. These polynomial facts were independently checked.

This is a counterexample to the *pruning criterion*, not a claim that the resulting quotient by $\sqrt{m}$ is the best final denesting, nor a claim that the automatic generator must propose this particular multiplier. It establishes that equal absolute degree is not a sound reason to rule out lower-degree GCD information.

11.2 Partial factorization is intentionally incomplete

`FactorInteger[n, Automatic]` extracts factors that are easy to find; it need not fully factor n [10]. The code then deletes composite residual factors (source lines 173–181). Potentially useful prime factors inside a residual composite are lost. Such a policy can be a reasonable speed tradeoff, but it needs to be reported as incomplete search, not as evidence that no denesting exists.

Further pruning removes some large, widely separated factors and uses only a selected prefix in the divisor grid. The discriminant itself is attached to the chosen polynomial representation; the code supplies no theorem converting this selection policy into complete coverage of all useful multipliers. In particular, the GCD-of-degrees branch is not supported by a proved relation between the corresponding fields.

11.3 The advertised multiplier cap is not a cap

Let r distinct primes be retained for (15). Its divisors have independent exponents $0, \dots, k-1$, so their number is exactly

$$k^r. \quad (42)$$

The code computes the selected-prime count approximately as

$$r = \min\left(\ell, \max\left(5, \left\lceil \frac{\log 1000}{\log k} \right\rceil\right)\right), \quad (43)$$

where ℓ is the available prime count after filtering (source lines 162–194). A ceiling usually makes k^r at least the target, and the minimum of five primes can make it vastly larger. With enough retained primes:

Root order k	Selected primes r	Divisors	After dropping 1
2	10	1,024	1,023
3	7	2,187	2,186
5	5	3,125	3,124
10	5	100,000	99,999
100	5	10,000,000,000	9,999,999,999

These are exact combinatorial counts, not measured trial counts. The union with additional discovered primes can add still more candidates. The variable `maxprimes = 14` is declared but never used.

A genuine bound for the grid alone would require $k^r - 1 \leq B$, with the remaining budget reserved for extra primes and existing proposals. Better still, generate exponent vectors lazily and stop after B accepted proposals. Calling `Divisors` and then truncating its output is not a memory-safe implementation of a cap, because the large list has already been materialized.

11.4 The subset pass adds another exponential mechanism

If q promising initial multipliers are selected, the `Distribute` construction can form up to 2^q subset products before duplicates are removed (source lines 402–420). The divisor-grid cap, even if repaired, would not bound

this separate expansion. The rational powers and coefficient heights can also increase the cost of a single minimal-polynomial, discriminant, or GCD computation enormously.

There is no global visited set of canonical multiplier keys, so the same algebraic multiplier can be proposed through different frames or in different syntactic forms. There is no explicit budget for total trials, total queued proposals, polynomial degree, coefficient size, time, or memory.

11.5 Termination and failure status need a real contract

The outer For loop compares its index with the current length of a worklist that can grow while it is being traversed. The source provides no monotone measure proving that this growth stops. The history often improves and the heuristics may work well in practice, but that is not a termination proof.

It would be inaccurate to say every `//` loop is completely unbounded: `ReplaceRepeated` has a documented default `MaxIterations` limit of 65,536 [13]. That built-in bound is not a global search budget, and hitting it is not a meaningful “no denesting exists” outcome. Kernel recursion or evaluation limits are similarly not substitutes for a deliberate algorithmic status.

The final messages distinguish mainly a changed expression from “no progress” (source lines 434–440). They do not distinguish search exhaustion, incomplete factorization, skipped equal-degree trials, failed algebraic operations, certification failure, and resource exhaustion. These are different outcomes and should be returned as data.

11.6 Sorting and simplification have hidden costs

`ComplexitySort` prioritizes `LeafCount` and uses `Abs` to break ties (source lines 500–504). This is a heuristic structural order, not a measure of algebraic field complexity or denesting quality. If exact comparisons remain unresolved, the ordering function may not provide the intended numerical ordering; forcing such comparisons can itself be costly. The initial sort by real part has a similar limitation outside the intended explicit-number domain.

`Repeated Expand, Together, Simplify`, discriminant factoring, and tree reconstruction add overhead before a useful multiplier is found. No benchmarks are supplied, so this report does not rank their empirical contribution. Caching algebraic computations and counting work by stage would make such an assessment possible.

12 Consolidated findings and repair priorities

Severity here describes consequences, not measured frequency. “High” includes silent value changes and uncontrolled resource growth; heuristic incompleteness is kept separate from soundness.

ID	Priority	Finding	Evidence and location
B1	Critical	Candidate selection targets $(\alpha^k)^{1/k}$ rather than α .	Algebraic counterexamples; 90–94, 125–127.
B2	Critical	Custom factorization uses unrestricted complex power laws.	Source rule and branch counterexamples; 527–543, 575–583.
B3	High	Non-fresh marker <code>f</code> can consume caller data.	Predicted source trace; 6, 55–73.
B4	High	Rationalizer’s <code>x -> 0</code> can erase a numerator named <code>x</code> .	Predicted source trace; 445–450.

ID	Priority	Finding	Evidence and location
B5	High	Empty candidate list is indexed as a success.	Unconditional access; 125–134.
B6	High	Multiplier “cap” admits exponential candidate lists.	Exact count k^r ; 162–199.
B7	High	No global work, time, memory, or queued-candidate limit.	Growing worklist and subset generation; 261–430.
B8	Medium	Prime comparator compares a pair with itself.	Wrong pure-function slot; 181.
B9	Medium	Loading removes <code>Print</code> protection without restoration.	Direct load-time side effect; 2.
B10	High	No consistent validation or failed-operation protocol.	Broad entry patterns, narrow trial pattern, positional assumptions; 79–134, 214–215.
H1	Medium	Equal-degree gate can skip a linear GCD.	Explicit algebraic counterexample; 115.
H2	Medium	Partial-factorization residuals are discarded.	Documented built-in behavior and source policy; 173–181.
H3	Medium	Discriminant and degree-GCD searches have no coverage proof.	Heuristic construction; 169–207.
H4	Medium	Initial “terms” may actually be factors or power arguments.	Head replacement; 224–229.
H5	Medium	No certified radical-form or complexity improvement criterion.	Success selection and replacement; 120–134, 309–324.
M1	Lower	Opaque list schemas, fixed factoring passes, and unused variables.	Maintenance and auditability; 95–103, 163, 253–267, 522.

Several source variables are unused or only diagnostic: `extrapower` is printed but not consumed, `maxprimes` is never applied, `expected` is unused inside a trial, `done` is unused inside the multiplier generator, and `best` in the extra-multiplier block is calculated but not used. The many commented-out alternatives and version-numbered function names suggest an experimental script, but they do not identify an author, development date, or historical correctness guarantee.

13 A safer design that preserves the useful mathematics

13.1 Separate proposal generation from proof

A robust implementation can retain the multiplier heuristic while treating it as an untrusted proposer. The acceptance layer should be much smaller and easier to audit. It must retain the original target, verify every proposed result exactly, and enforce the chosen output language and improvement criterion.

The required invariants are:

1. The semantic target never changes. A powered radicand is auxiliary data; it does not replace the original input

for equality testing.

2. Every transformation used as an equality either has proved branch conditions or passes exact verification against its own pre-transformation value.
3. A candidate is called “denested” only after exact equality and the output representation criterion both pass.

These checks belong around the entire wrapper, not only around one internal trial. For exact numeric inputs, a final check against the original `expr` can catch preprocessing errors. For symbolic inputs, either require a clearly specified assumption system with proved transformations or reject them explicitly.

13.2 Use exact algebraic verification conservatively

For explicit exact algebraic numbers, one can screen cheaply and then certify by reducing the difference to zero. A failed or timed-out verification should not be reinterpreted as equality. The strengthened acceptance condition is

$$\text{RootReduce}(\gamma - \alpha) = 0, \quad (44)$$

with the actual original target α .

An acceptance layer should also reject `Indeterminate`, infinities, unresolved variables, approximate numbers, `ConditionalExpression` objects outside the contract, and failed solver calls. `TrueQ` is useful where a total Boolean decision is required, but replacing every test by `TrueQ` does not turn an undecidable or unevaluated expression into a proof; it merely makes failure conservative.

13.3 Choose a denesting objective explicitly

One possible objective is a lexicographic score built from radical nesting height, number of rational-power nodes, and expression size. It should be accompanied by an output-language predicate that disallows `Root` or `AlgebraicNumber` when the task specifically requires radicals. Otherwise a canonical algebraic number representation can win a syntactic score without satisfying the intended mathematical goal.

A candidate may be exactly equal but not improved. One suitable status is `"EqualButNotImproved"`. It should not be described as `"Denested"`. When several certified expressions are available, compare their representation scores instead of taking the first structurally distinct expression returned by `Solve`.

13.4 Make state and resources explicit

Use associations or another tagged record type for attempts and work items. For example, distinguish

```
<|"Status" -> "NoLowerDegreeGCD", "Degree" -> d|>
<|"Status" -> "CertifiedCandidates", "Candidates" -> values|>
Failure["NoCertifiedCandidate", <|"Multiplier" -> m|>]
Failure["TimeLimit", <|"TrialsCompleted" -> count|>]
```

A deterministic queue, a visited set, lazy candidate generation, and an actual trial budget should replace the implicit growth policy. Include the root order, coefficient-field context, and any other relevant trial state in cache keys. Canonicalization can identify algebraically identical multipliers, but it also has a cost and must be budgeted.

For pure rational multipliers, reducing prime valuations modulo k can sometimes organize equivalent search classes. Such an optimization needs a proof appropriate to the trial and branch conventions; it should not be silently inferred from the current exponent-reduction rules.

13.5 A guarded one-multiplier step

The archive includes `safer_core.wl`. It is a deliberately limited reference component, not a patched replacement for all 585 source lines. It validates exact algebraic input, validates $k \geq 2$ and $m \neq 0$, computes the same GCD idea without absolute-degree pruning, enumerates roots of unity, certifies against the original target, handles an empty candidate set, and returns structured results under a time limit. Its key acceptance block is

```
roots = t /. rules;
candidates = Flatten[Outer[Times,
  roots/multiplier^(1/k),
  Exp[2 Pi I Range[0, k - 1]/k]]];
accepted = Select[candidates,
  exactAlgebraicQ[#] && ExactAlgebraicEqualQ[#, target] &];
If[accepted == {},
  Return[Failure["NoCertifiedCandidate", <|
    "Multiplier" -> multiplier, "GCDDegree" -> dg|>]]];
```

The complete component is supplied as source, with an explicit warning that it was not executed in a Wolfram kernel during this audit. It certifies equality, not radical denesting; it can still return a representation that a higher-level radical-only policy would reject. It also does not implement a complete search or a global memory budget. Keeping these boundaries explicit is preferable to calling a small illustrative repair a production-ready denester.

14 Verification performed and proposed regression testing

14.1 Independently executed checks

The included `verify_algebra.py` script was executed successfully. It uses SymPy for exact minimal polynomials, polynomial GCDs over explicit algebraic extensions, polynomial division, and identity checks; it uses mpmath at 80 decimal digits to check the principal-branch examples. Its text output and JSON output are included.

The checks cover the positive worked example, the negative branch example, the complex $3/2$ power, the merged square-root product, the equal-absolute-degree pruning counterexample, three denominator-rationalization examples, the two invalid power identities, the malformed comparator's logic, and exact multiplier-grid counts. Numerical results supplement the exact arguments rather than replacing them.

These checks validate the algebra used in the audit. They do not establish the complete behavior of `FactorList`, `Solve`, evaluation scoping, expression canonicalization, or the mutable search stack in a specific Wolfram release.

14.2 Wolfram tests supplied but not executed

The file `regression_tests.wlt` contains expected-behavior tests for the original source and a few tests of the guarded component. Several expectations are deliberately preservation requirements that the legacy code is predicted to violate; they are not presented as a passing legacy suite.

The suite checks that loading preserves `Print` attributes, that the prime comparator compares the two prime values, that negative and complex branches are preserved, that a symbolic square is not flattened incorrectly, that a caller's `f` is not consumed, and that a numerator named `x` survives rationalization. Positive examples and all-level value preservation are included as controls. The guarded component is tested for rejection of a zero multiplier and invalid root order and for preservation of negative and complex targets.

Because loading the original file changes `Print`, these tests should be run in a disposable, fresh kernel. The suite records and restores the prior attribute list after loading. A typical invocation from the archive directory is

```
wolframscript -code 'TestReport["regression_tests.wlt"]'
```

Record `$Version`, `$SystemID`, `$Assumptions`, relevant evaluation limits, and the test report alongside results. A failure caused by automatic pre-evaluation must be interpreted against the actual input reaching the function, not assumed to invalidate the mathematical branch counterexample.

14.3 Tests needed beyond the supplied regression file

A broader suite should cover conjugate inputs, branch-cut boundary values, negative rational exponents, mixed root orders, integer powers containing radicals, zero and singular expressions, symbolic and approximate inputs, invalid callbacks, and repeated or empty multiplier lists. It should test every public entry point, not only `denest11`.

Useful properties include exact value preservation, no leakage of private markers, no mutation of unrelated definitions or attributes, stability under renaming unrelated symbols, and explicit resource-limit results. Idempotence is desirable for a normalizer, but should not be imposed as a theorem on an intentionally budget-limited heuristic without specifying how the budget is used.

A performance harness should separately record preprocessing time, minimal- polynomial time, GCD time, factorization time, number of unique trials, peak queued candidates, output score, and exact-verification time. No performance improvement percentage is claimed here because none was measured.

15 Conclusion

The program contains a substantive algebraic search idea. Its use of a minimal polynomial together with an algebraic-coefficient GCD can reveal a simpler relation, and its enumeration of roots of unity is a mathematically appropriate way to recover possible branches. Its denominator-rationalization formula is also valid for nonzero algebraic denominators.

Those strengths do not make the implementation a reliable general denester in its current form. The original target is not preserved during candidate selection; the custom factorizer performs branch-sensitive rewrites without conditions; non-fresh temporary symbols can alter caller expressions; and several error paths and resource limits are missing. The sorting typo and the ineffective multiplier cap are concrete implementation defects, while partial factorization and unproved degree-based pruning explain why a failed search says little about the existence of a denesting.

The first repair priority is correctness, not a larger multiplier search: retain the original expression, use branch-safe preprocessing, make temporary symbols fresh, validate operations, and require exact equality before accepting anything. Only then should the search heuristics be optimized and benchmarked. With that separation, the current multiplier strategy can be retained as one useful proposal engine without being asked to serve as its own proof of correctness or completeness.

References

- [1] Supplied attachment, *Pasted text.txt*. Original bytes preserved as `original.wl`; SHA-256 stated on the title page. Source reproduced in Appendix A. No author or version metadata supplied.
- [2] Wolfram Research, *MinimalPolynomial*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/MinimalPolynomial.html>.

- [3] Wolfram Research, *PolynomialGCD*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/PolynomialGCD.html>.
- [4] Wolfram Research, *RootReduce*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/RootReduce.html>.
- [5] Wolfram Research, *Power*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/Power.html>.
- [6] Wolfram MathWorld, *Principal Root*. <https://mathworld.wolfram.com/PrincipalRoot.html>.
- [7] Wolfram Research, *Block*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/Block.html>.
- [8] Wolfram Research, *Module*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/Module.html>.
- [9] Wolfram Research, *PossibleZeroQ*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/PossibleZeroQ.html>.
- [10] Wolfram Research, *FactorInteger*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/FactorInteger.html>.
- [11] Wolfram Research, *Sort*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/Sort.html>.
- [12] Wolfram Research, *Function*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/Function.html>.
- [13] Wolfram Research, *ReplaceRepeated*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/ReplaceRepeated.html>.
- [14] Wolfram Research, *MemberQ*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/MemberQ.html>.
- [15] Wolfram Research, *ComplexExpand*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/ComplexExpand.html>.
- [16] Wolfram Research, *Unprotect*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/Unprotect.html>.
- [17] Wolfram Research, *FactorList*, Wolfram Language Documentation. <https://reference.wolfram.com/language/ref/FactorList.html>.

All external documentation above was consulted on 5 September 2026. The source listing and original analysis, rather than an external denesting algorithm or an unseen specification, are the basis of the audit.

A Complete supplied source, with physical line numbers

The following listing is the supplied program, not a corrected version. Line numbers refer to the original physical lines, including blank lines. Long lines may wrap in the PDF without receiving a new line number. The file in the archive preserves the original CRLF byte stream and its SHA-256 fingerprint.

```

1 ClearAll[Strad];
2 Unprotect[Print];
3 Strad[eaf_] := Block[{Print = (Null &)}, DenestRadicals3[eaf]];
4 ClearAll[DenestRadicals3];
5 DenestRadicals3[expr_, alllevels_ : False, func_ : denest11] :=
6   Block[{f, mod, level, problems, i1, i2},
7     If[alllevels != True,
8       level = ReplaceAll,
9       level = ReplaceRepeated
10    ];
11    mod = level[Factorc[expr],
12      radicand_?AlgebraicQ^pow_Rational := Block[{factored, SumQ, i, j},
13        SumQ[expression_] :=
14          MemberQ[
15            expression, _Plus | Complex[Except[0], _], {0, Infinity}];
16        factored = FactorTermsList[radicand];
17        factored[[2]] =
18          Flatten[{Replace[Factorc[factored[[2]]],
19            prod_Times := List @@ prod}]];
20        factored[[1]] =
21          DeleteCases[{#, ComplexExpand[factored[[1]]/#]} &[
22            GCD[Re[factored[[1]]], Im[factored[[1]]]], 1];
23        factored[[1]] = #[[1]]^#[[2]] & /@
24          Flatten[FactorInteger /@ factored[[1]], 1];
25        factored = Flatten[factored];
26        factored =
27          factored //.
28          list : {a___, expr1_, b___, expr2_, c___} := {a, b, c,
29            ComplexExpand[expr1*expr2]} /; And[SumQ[expr1], SumQ[expr2]];
30        j = 1;
31        For[{i = 1}, i <= Length[factored], i++,
32          If[N[Re[factored[[i]]], 100] < 0,
33            factored[[i]] = -factored[[i]];
34            j *= -1
35          ];
36        ];
37        factored = DeleteCases[Prepend[factored, j], 1];
38        f[Simplify[
39          Times @@ (Simplify[RationalizeDenominator[Together[#]]] & /@
40            factored)], pow]
41      ]
42    ];
43    mod = mod //.
44    prod : Times[a___, expr1 : f[rad1_, pow1_], b___,
45      expr2 : f[rad2_, pow2_], c___] :=
46      Times[a, b, c, f[(rad1^pow1)*(rad2^pow2)]] /;
47      And[! FreeQ[rad1, Power], ! FreeQ[rad2, Power]];
48    mod = level[mod, f[rad_, pow_] := f[rad^pow]];
49    mod = mod //.
50    prod : Times[a___, expr1 : f[rad1_], b___, expr2 : f[rad2_],
51      c___] :=
52      Times[a, b, c, f[rad1*rad2]] /;
53      And[And[Re[#] != 0, Im[#] != 0] &[N[rad1, 10]],
54      And[Re[#] != 0, Im[#] != 0] &[N[rad2, 10]]];
55    problems = DeleteDuplicates[Cases[mod, f[_], {0, Infinity}]];
56    Print[problems];
57    Print[];
58    If[alllevels != True,
59      mod =
60        Simplify[
61          mod /. ((# -> Replace[#, f[val_] := func[val]]) & /@ problems)],
62      For[{i1 = 1}, MemberQ[problems, f[_]] == True, i1++,

```

```

63   i1 = Mod[i1, Length[problems], 1];
64   If[MatchQ[problems[[i1]], f[problem_] /; FreeQ[problem, f[...]]],
65     problems[[i1]] = problems[[i1]] -> (problems[[i1]] /. f -> func);
66   For[{i2 = 1}, i2 <= Length[problems], i2++,
67     If[Head[problems[[i2]]] != Rule,
68       problems[[i2]] = problems[[i2]] /. problems[[i1]];
69     ];
70   ];
71 ];
72 ];
73 mod = Simplify[mod /. problems];
74 ];
75 Return[mod];
76 ]
77
78 ClearAll[denest11];
79 denest11[problem_, forcedmultipliers_ : {}] :=
80 Block[{radicand, outerpowers, reductionpower, extrapower,
81   multiplierstack, multipliers, multiplier, terms, history, attempt,
82   solution, updatehistory, stackchange, addmultipliers,
83   trymultiplier, newmultipliers, x, i1, i2, i3},
84   If[! MemberQ[problem,
85     Power[inner_?(Function[var, MemberQ[var, Power[a_, b_], Infinity]]],
86     outer_, {0, Infinity}],
87     Return[problem];
88   ];
89   Print["problem ", problem];
90   reductionpower = (LCM @@ (maxnestdepth[problem,
91     MatchQ[#, Power[_Rational]] &] /. {Power[base_, exp_, _] ->
92     Denominator[exp]}));
93   outerpowers = 1/reductionpower;
94   radicand = problem^reductionpower;
95   extrapower =
96     Times @@ ((LCM @@ #) & /@ (DeleteCases[
97     Cases[nestdepthsort[radicand,
98       MatchQ[#, Power[_Rational]] &],
99     patt : {{expr_, nest_}, Repeated[{_, _}, {0, Infinity}]} /;
100     nest >= 2], {Except[Power[_Rational]], _, {2}} /.
101     patt : {Power[base_, exp_, nest_] -> Denominator[exp]}));
102   Print["outer root: ", reductionpower, " extra root: ", extrapower];
103   Print[];
104
105   (*tries a multiplier
106   returns {<boolean progress>, {multiplier, minpoly, degree}, [answer]}*)
107   trymultiplier[radicand_, outerpowers_, multiplier_, best_,
108     worseknown_] /; MatchQ[outerpowers, Rational[1, _Integer]] :=
109   Block[{minpoly, degree, polygcd, possibilities, result, expected,
110     arg1},
111     minpoly = MinimalPolynomial[(multiplier*radicand)^outerpowers];
112     degree = Exponent[minpoly[arg1], arg1];
113     result = {False, {multiplier, minpoly, degree}};
114     Print["multiplier: ", multiplier, " minpoly degree: ", degree];
115     If[Or[degree < best, GCD[degree, best] < best],
116       polygcd =
117         PolynomialGCD[minpoly[arg1],
118         arg1^(1/outerpowers) - multiplier*radicand,
119         Extension -> Automatic];
120       If[And[Exponent[polygcd, arg1] < (1/outerpowers), polygcd != 1],
121         possibilities =
122           Flatten[Outer[Times,
123             arg1/(multiplier^outerpowers) /. Solve[polygcd == 0, arg1],
124             Exp[2*I*Pi*Range[0, (1/outerpowers) - 1, 1]/(1/outerpowers)]]];
125       result =
126         DeleteDuplicates[
127           Select[possibilities, PossibleZeroQ[# - radicand^outerpowers] &]];
128       If[Length[result] > 1,
129         Print[Length[result], " possible solutions, taking the first"];
130       Print[result];
131       result = Take[result, 1];
132     ];

```

```

133 (*progress because a denesting occurred*)
134 result = {True, {multiplier, minpoly, degree}, result[[1]]},
135 If[polygcd != 1,
136   If[
137     Or[degree < best,
138       And[GCD[degree, best] < best, worseknown != True]],
139     (*progress because degree of minpoly has decreased*)
140     result = {True, {multiplier, minpoly, degree}}
141   ],
142   Print[
143     "PolynomialGCD error: ", {radicand, outerpowers, multiplier,
144       minpoly}]
145   ],
146 ],
147 If[And[worseknown != True, degree != best],
148   (*progress because this multiplier has greater degree than a pre\
149   vious multiplier*)
150   Print["previous progress identified"];
151   result = {True, {multiplier, minpoly, degree}};
152   ]
153 ];
154 Return[result];
155 ];
156
157 (*form for attempts is {multiplier, minpoly, degree},
158 attempt2 is Null if only one attempt is available*)
159 newmultipliers[smaller_, larger_, reductionpower_] :=
160 Block[{done, new, primes, multiplierprimes, minprimes, maxprimes,
161   primeratio, multipliercap, arg1},
162   minprimes = 5;
163   maxprimes = 14;
164   primeratio = 100;
165   multipliercap = 1000;
166
167   (*two possibilites: reduction in degree,
168   reduction only in GCD of degrees*)
169   If[Or[larger == Null,
170     GCD[smaller[[-1]], larger[[-1]]] ==
171     Min[smaller[[-1]], larger[[-1]]],
172     (*one degree is a multiple of the other, usual case*)
173     primes =
174     DeleteCases[
175       FactorInteger[Discriminant[smaller[[2]][arg1], arg1],
176         Automatic], {-1, 1}];
177     primes =
178     Sort[DeleteCases[primes,
179       factor : {prime_, _} /;
180       And[! PrimeQ[prime], (Print["composite factor: ", factor];
181         True)], #1[[1]] <= #2[[2]] &];
182     primes = Flatten[primes[All, 1]];
183     multiplierprimes = primes;
184     For[{i3 = Length[primes]},
185       And[i3 > minprimes,
186         multiplierprimes[[i3]]/multiplierprimes[[i3 - 1]] > primeratio],
187       i3--,
188       multiplierprimes[[i3]] = 0;
189     ];
190     multiplierprimes =
191     DeleteCases[multiplierprimes, 0][[
192       1 ;; Min[
193         Max[minprimes, Ceiling[Log[multipliercap]/Log[reductionpower]]],
194         Length[multiplierprimes]]]];
195     new =
196     smaller[[1]]*
197     Drop[Union[
198       Divisors[(Times @@ multiplierprimes)^(reductionpower - 1)],
199       primes], 1];
200   ],
201   (*different degree reductions in each degree from a larger degree\
202   *)

```

```

203     Print[
204         "GCD of smallest degrees is smaller than any observed degree: \
205 ", {smaller[{{1, -1}}], larger[{{1, -1}}]}}];
206     new = smaller[[1]]*larger[[1]]*{1};
207     Print[PolynomialGCD @@ {smaller[[2]][x], larger[[2]][x]}};
208 ];
209     new = {{False, smaller}, ComplexitySort[new]};
210     Return[new];
211 ];
212
213 (*compute initial set of multipliers*)
214 If[ListQ[forcedmultipliers],
215     multipliers = {{False, {0, Infinity, Infinity}}, forcedmultipliers},
216     (*old, may be better*)
217     (*
218     terms=DeleteCases[Replace[#, {Times[a___,
219     Alternatives[_Rational, _Integer], b___]\[RuleDelayed]a*b,
220     Alternatives[_Rational, _Integer]\[Rule]0, Complex[a_,
221     b_]\[RuleDelayed]Sign[b]*I}]/@ (List @@ Expand[
222     RationalizeDenominator[radicand]]), 0];
223     *)
224     terms =
225     DeleteCases[
226     Replace[#, {Times[a___, Alternatives[_Rational, _Integer],
227     b___] :> a*b, Alternatives[_Rational, _Integer] -> 0,
228     Complex[a_, b_] :> Sign[b]*I}]/@ (List @@
229     Expand[RationalizeDenominator[radicand]]), 0];
230     multipliers = Prepend[Replace[If[Head[#] == Times,
231
232         Times @@ (Function[{var},
233             var^(maxnestdepth[var,
234                 Function[{var2},
235                     MatchQ[var2, Power[_ , _Rational]]][[
236                     1]] /. {{Power[base_,
237                     exp_] :> (Denominator[exp] - Numerator[exp])/
238                     Numerator[exp], {Complex[
239                     0, _], _} :> -1}}] /@ (List @@ #)),
240             #^(maxnestdepth[#,
241                 Function[{var}, MatchQ[var, Power[_ , _Rational]]][[
242                     1]] /. {{Power[base_,
243                     exp_] :> (Denominator[exp] - Numerator[exp])/
244                     Numerator[exp], {Complex[0, _], _} :> -1}}
245             ],
246             Times[a___, Alternatives[_Rational, _Integer], b___] :>
247             a*b] & /@ Sort[terms, (Re[#1] <= Re[#2]) &], 1];
248     multipliers = {{True, {0, Infinity, Infinity}}, multipliers};
249 ];
250
251 solution = problem;
252 history = {{0, Infinity, Infinity}};
253 (*multiplierstack form {{<do full set>,{<multiplier,minpoly,
254 degree for generating this set>},{<multipliers>}},...}*)
255 multiplierstack = {multipliers};
256 (*add power set of multipliers as second element in multiplierstack*)
257 (*multiplierstack=Append[multiplierstack,
258 {multiplierstack[[1,1]],ComplexitySort[DeleteCases[Union[(Times@@#)&/@
259 Subsets[DeleteCases[multipliers[[2]],1]]],mult_/;MemberQ[
260 multipliers[[2]],mult]]]}];*)
261 Catch[
262 For[{i1 = 1}, i1 <= Length[multiplierstack], i1++,
263     (*compute new multipliers from data if not already done*)
264     If[ListQ[multiplierstack[[i1, 2]]] != True,
265         multiplierstack[[i1]] =
266         multiplierstack[[i1, -2]] @@ (multiplierstack[[i1, -1]]);
267     ];
268     multipliers = multiplierstack[[i1]];
269     stackchange = False;
270     If[multipliers[[1, 2]] != history[[1]],
271         Print["multiplier count: ", Length[multipliers[[2]]], " from: ",
272             multipliers[[1, 2, {1, -1}]]],

```

```

273 If[! ListQ[forcedmultipliers],
274   Print["multiplier count: ", Length[multipliers[[2]]],
275   " multipliers: ", multipliers[[2]]];,
276   Print["multiplier count: ", Length[multipliers[[2]]];
277   ]
278 ];
279 For[{i2 = 1}, i2 <= Length[multipliers[[2]]], i2++,
280   multiplier = multipliers[[2, i2]];
281   updatehistory = False;
282   addmultipliers = False;
283   (*try a multiplier, different based on history*)
284   If[And[Length[history] >= 3, history[[-2]] != history[[1]]],
285     attempt =
286     trymultiplier[radicand, outpower, multiplier,
287     history[[-1, -1]], True],
288     attempt =
289     trymultiplier[radicand, outpower, multiplier,
290     history[[-1, -1]], False];
291   ];
292   multiplierstack[[i1, 2, i2]] = {multiplier, attempt[[2, -1]]};
293
294   If[And[multipliers[[1, 1]] === True,
295     attempt[[2, -1]] <= history[[-1, -1]]],
296     addmultipliers = True
297   ];
298   (*check if progress has been made on this attempt,
299   no action taken if no progress*)
300   If[attempt[[1]] === True,
301     (*history will be updated*)
302     updatehistory = True;
303
304     (*decide if a stackchange will be made*)
305     If[history[[-1]] != history[[1]],
306       stackchange = True;
307     ];
308
309   If[ListQ[attempt[[-1]]] != True,
310     (*a solution was obtained*)
311     Print["success: ",
312     Join[attempt[[2, {1, -1}]],
313     If[IntegerQ[multiplier], {FactorInteger[multiplier]}, {}]]];
314     solution = Simplify[attempt[[-1]]];
315     If[
316     Or[history[[-1]] === history[[1]],
317     history[[2, -1]]/attempt[[2, -1]] >= reductionpower],
318     (*not worth trying for additional denesting, exit immediately*)
319     history = Append[history, attempt[[2]]];
320     Throw["solution"],
321     (*further denesting will be attempted*)
322     Print["attempting additional denesting, current solution is:"];
323     Print[solution];
324   ];
325   ];
326
327   If[
328     And[history[[-1]] != history[[1]],
329     Or[attempt[[2, -1]] < history[[-1, -1]],
330     history[[-2]] === history[[1]]],
331     (*improvement has occurred, new multipliers will be created*)
332     Print["improvement ", attempt[[2, {1, -1}]]];
333     addmultipliers = True;
334   ];
335   ];
336
337   (*add new multipliers if needed*)
338   If[addmultipliers === True,
339     Block[{smaller, larger, done, new},
340       If[attempt[[2, -1]] <= history[[-1, -1]],
341         smaller = attempt[[2]];
342         larger = history[[-1]],

```

```

343     smaller = history[[-1]];
344     larger = attempt[[2]];
345 ];
346 new = {{False, smaller},
347     newmultipliers, {smaller,
348         If[larger != history[[1]], larger, Null], reductionpower}};
349
350 Print["adding new multipliers from: ", multiplier];
351 (*update multiplierstack with new multipliers*)
352 If[multipliers[[1, 1]] == False,
353     (*use new multipliers immediately*)
354     done = {multipliers[[1]], multipliers[[2, 1 ;; i2]]};
355     multiplierstack = Insert[multiplierstack, new, i1 + 1];
356     If[i2 < Length[multipliers[[2]]],
357
358         multiplierstack =
359             Insert[multiplierstack, {multipliers[[1]],
360                 multipliers[[2, i2 + 1 ;; -1]]}, i1 + 2];
361     ];
362     multiplierstack[[i1]] = done,
363     (*use new multipliers later*)
364     stackchange = False;
365     For[{i3 = Length[multiplierstack]}, i3 >= i1, i3--,
366
367         If[Or[multiplierstack[[i3, 1, 2]] == history[[1]],
368             new[[1, 2, -1]] > multiplierstack[[i3, 1, 2, -1]],
369             And[new[[1, 2, -1]] == multiplierstack[[i3, 1, 2, -1]],
370                 LeafCount[new[[1, 2, 1]]] >=
371                 LeafCount[multiplierstack[[i3, 1, 2, 1]]]],
372             multiplierstack = Insert[multiplierstack, new, i3 + 1];
373             Break[];
374         ]
375     ];
376 ];
377 ]
378 ];
379
380 (*update history if needed*)
381 If[updatehistory == True,
382     For[{i3 = Length[history]}, i3 >= 1, i3--,
383         If[history[[i3, -1]] > attempt[[2, -1]],
384             history = Insert[history, attempt[[2]], i3 + 1];
385             Break[];
386         ]
387     ];
388 ];
389
390 (*break inner loop if stack has changed*)
391 If[stackchange == True,
392     Break[];
393 ];
394 ];(*end of inner loop*)
395
396 (*add additional multipliers after first pass (if no forced multip\
397 liers)*)
398 If[And[i1 == 1, ! ListQ[forcedmultipliers]],
399     Block[{best, worst, extra, f},
400         best = Min[multiplierstack[[i1, 2, All, 2]]];
401         worst = Max[multiplierstack[[i1, 2, All, 2]]];
402         extra =
403             DeleteCases[
404                 DeleteDuplicates[
405                     Replace[Distribute[
406                         f @@ (#^Range[0, 1] & /@ (Cases[
407                             multiplierstack[[i1, 2]], {mult_, degree_} /;
408                             degree < worst][[All, 1]])),
409                         List] /. {f[a___, Power[base_Times, exp_], b___] :=>
410                             f[a, b, Sequence @@ ((List @@ base)^exp)],
411                             f[a___, prod_Times, b___] :=> f[a, b, Sequence @@ prod]},
412                         f[a___, Power[base_, exp_], b___] :=>

```

```

413      f[a, b, (-1)^exp, (-base)^exp] /;
414      And[base != -1, Re[base] < 0],
415      f[a___, Power[base_, Rational[num1_, denom_]], b___,
416      Power[base_, Rational[num2_, denom_]], c___] :>
417      f[a, b, c, base^(Mod[num1 + num2, denom]/denom)]] /.
418      f -> Times[a___, _Integer, b___] :> a*b, {1}] //.
419      Power[base_, Rational[a_, b_]] :> base^(Mod[a, b]/b)],
420      Alternatives @@ multiplierstack[[i1, 2, All, 1]]];
421      If[Length[extra] >= 1,
422      Print["adding multipliers based on progress in initial guess"];
423      multiplierstack =
424      Insert[multiplierstack, {multiplierstack[[i1, 1]], extra},
425      i1 + 1];
426      ];
427      ]
428      ];
429
430      ](*end of outer loop*),
431      "solution"
432      ];(*end of Catch, used to break the loop when a solution is found*)
433
434      Print["history: ", #[[{1, -1}]] & /@ history];
435      If[solution != problem,
436      Print[solution],
437      Print["no progress"];
438      ];
439      Print[];
440      Return[solution];
441      ]
442
443      ClearAll[RationalizeDenominator];
444      RationalizeDenominator[expr_] :=
445      Block[{conjugate, denom, x},
446      denom = Denominator[expr];
447      conjugate = MinimalPolynomial[denom];
448      Expand[
449      Numerator[expr]*PolynomialQuotient[conjugate[x], x - denom, x] /.
450      x -> 0]/(-1*conjugate[0])
451      ]
452
453      ClearAll[AlgebraicQ];
454      AlgebraicQ[num_] := Element[num, Algebraics]
455
456      ClearAll[nestdepths]
457      nestdepths[expr_, func_] :=
458      Block[{inner},
459      If[Depth[expr] === 1,
460      {expr, If[func[expr], 1, 0]},
461      inner = Map[nestdepths[#, func] &, (List @@ expr), {1}];
462      {expr,
463      Sequence @@
464      inner, ((Max @@ ({#[[-1]]} & /@ inner)) + If[func[expr], 1, 0])}
465      ]
466      ]
467
468      ClearAll[nestdepthsort]
469      nestdepthsort[expr_, func_] :=
470      Block[{data},
471      data = nestdepths[expr, func];
472      data =
473      data //.
474      patt : {a : Except[_List], b : __, c : Except[_List]} :> {{a, c},
475      b};
476      data = data //. patt : {rep : Repeated[_], _} :> Sequence @@ patt;
477      data = Gather[{data}, (#1[[2]] == #2[[2]]) &];
478      data = Sort[data, (#1[[1, 2]] >= #2[[1, 2]]) &];
479      data
480      ]
481
482      ClearAll[maxnestdepth]

```

```

483 maxnestdepth[expr_, func_] :=
484   Block[{data, max},
485     data = nestdepths[{expr}, func][[2]];
486     max = data[[-1]];
487     data =
488       FixedPoint[
489         DeleteCases[# /. {a_, b_} :> {} /; b < max, {}, Infinity] &, data];
490     data =
491       If[And[ListQ[#], Length[#] > 2], Sequence @@ (#[[2 ;; -2]]), #] & /@
492         data;
493     If[Length[data] > 2,
494       data = data[[2 ;; -2]],
495       data = {data}
496     ];
497     data
498   ]
499
500 ClearAll[ComplexitySort];
501 ComplexitySort[expr_] :=
502   Return[Sort[expr,
503     Switch[LeafCount[#1] - LeafCount[#2], a_ /; a < 0, True,
504       a_ /; a > 0, False, 0, Abs[#1] <= Abs[#2]] &]];
505
506 ClearAll[Factorrc]
507 Factorrc[expr_] :=
508   Flatten[
509     fullfactor[
510       expr] /. {pow : {a : Except[_List], b : Except[_List]} :> {a^b},
511       prod : {rep : (Repeated[{Except[_List]], {2,
512         Infinity}})} :> {Times[rep]},
513       sum : {rep : Repeated[{Except[_List]], {2, Infinity}}]} :>
514       Plus[rep],
515       pow2 : {{a : Except[_List], b : Except[_List]} :> {a^b},
516       pow3 : {{a : Except[_List], b : Except[_List]} :> {a^b}}][[1]]
517   ]
518 ClearAll[fullfactor]
519 fullfactor[expr_] :=
520   Block[{factorlist, i = 0, j = 0, done = False},
521     factorlist = FactorList[expr];
522     While[! done && j < 5,
523       j++;
524       done = True;
525       For[{i = 1}, i <= Length[factorlist], i++,
526         Which[
527           Head[factorlist[[i, 1]]] === Power,
528           done = False;
529           factorlist[[
530             i]] = {{factorlist[[i, 1, 1]],
531               factorlist[[i, 1, 2]]*factorlist[[i, 2]]}},
532           And[
533             MemberQ[{Integer, Rational}, Head[factorlist[[i, 1]]], !
534             PrimeQ[factorlist[[i, 1]], Abs[factorlist[[i, 1]]] != 1],
535           done = False;
536           factorlist[[i]] = {#[[1]], #[[2]]*factorlist[[i, 2]]} & /@
537             FactorInteger[factorlist[[i, 1]]],
538           Head[factorlist[[i, 1]]] === Plus,
539           done = False;
540           factorlist[[i]] =
541             combine[fullfactor /@
542               Replace[factorlist[[i, 1]], Plus[a_, b_] :> {a, b}],
543             factorlist[[i, 2]]],
544           True,
545           factorlist[[i]] = {factorlist[[i]]}
546         ]
547       ];
548     factorlist = Flatten[factorlist, 1]
549   ];
550   factorlist
551 ]
552

```

```

553 ClearAll[combine];
554 combine[terms_List, pow_] :=
555 Block[{simpler = terms, base, bases, positions, factor, i, j, k},
556   For[{i = 1}, i <= Length[terms], i++,
557     base = terms[[i]];
558     For[{j = 1}, j < Length[base], j++,
559       For[{k = j + 1}, k <= Length[base], k++,
560         If[base[[j, 1]] == base[[k, 1]],
561           base[[j]] = {base[[j, 1]], base[[j, 2]] + base[[k, 2]]};
562         base[[k]] = {0, 0}
563       ]
564     ];
565     If[base[[j, 1]] == 1,
566       base[[j]] = {0, 0}
567     ];
568   ];
569   simpler[[i]] = Delete[base, Position[base, {0, 0}]];
570 ];
571 bases = Intersection @@ Map[#[[1]] &, simpler, {2}];
572 For[{i = 1}, i <= Length[bases], i++,
573   positions =
574     Position[#, {bases[[i]], Except[_List]}, 1] & /@ simpler;
575   factor =
576     Min[Extract[simpler[[#]], positions[[#]]][[1, 2]] & /@
577       Range[1, Length[simpler], 1]];
578   For[{j = 1}, j <= Length[simpler], j++,
579     simpler[[j, Sequence @@ (positions[[j, 1])], 2]] -= factor
580   ];
581   bases[[i]] = {bases[[i]], factor*pow};
582 ];
583 Append[bases, {simpler, pow}]
584 ]

```

B Archive contents and reproducibility

The requested deliverables are `article.tex` and `article.pdf`. The additional files make the reasoning and proposed runtime checks auditable:

File	Purpose
<code>original.wl</code>	Original attachment, byte-for-byte.
<code>verify_algebra.py</code>	Executed independent algebra and branch checks.
<code>verification_results.txt</code>	Captured output of that execution.
<code>verification_results.json</code>	Machine-readable independent check results.
<code>safer_core.wl</code>	Statically reviewed, unexecuted guarded trial component.
<code>regression_tests.wlt</code>	Unexecuted Wolfram expected-behavior tests.
<code>README.md</code>	Build and test instructions, plus the execution boundary.
<code>SHA256SUMS.txt</code>	Checksums of archive content other than the checksum file.

Rebuild the article from this directory with

```
pdflatex -interaction=nonstopmode -halt-on-error article.tex
pdflatex -interaction=nonstopmode -halt-on-error article.tex
```

The original source file is required alongside the LaTeX file because the full listing is included with `\lstinputlisting`. No external image or font file is needed. Re-run the independent checks with

```
python verify_algebra.py
```

The Python script requires SymPy and mpmath. It does not require, contact, or substitute for a Wolfram kernel. Re-running the Wolfram tests is a separate step.